Decoupled Information-Set MCTS with a Card-Value Model for Sushi Go!

Halit Ozger
Student no. [to be completed]
Queen Mary University of London
London, UK

[Group member 2]
Student no. [to be completed]
Queen Mary University of London
London, UK

[Group member 3]
Student no. [to be completed]
Queen Mary University of London
London, UK

***Abstract*—In *Sushi Go!*, players pick cards simultaneously from hidden hands that are passed on, which basic Monte Carlo tree search (MCTS) does not model. To BasicMCTS of the Tabletop Games (TAG) framework we added rollouts guided by a card-value model, a lead-based evaluation, round-end rollouts, max-n backups, decoupled selection and information-set search. On held-out deals the agent wins 52.6%, 47.2% and [pending]% of 500 three-player games at 100, 500 and 900 ms against TAG's information-set MCTS and our previous agent (chance 33.3%), and 44.4% of four-player games at 100 ms (chance 25%). In leave-one-out ablations every component improves the score margin significantly, the evaluation and the decoupled information-set tree also the win rate; settings tuned for four players cost 7.7 percentage points with three (exploratory).**

Keywords*—*Monte Carlo tree search, information sets, simultaneous moves, card games, Sushi Go!, TAG

I. Introduction

Tabletop games combine chance, hidden information and several interacting players within small rule sets; the Tabletop Games framework (TAG) [1] gives many of them a common interface so that agents can be compared. Monte Carlo tree search (MCTS) [2] is the usual starting point for such agents, but its basic form assumes that the state is fully observed and that players move one after another [3]. *Sushi Go!*, one of TAG's games, breaks both assumptions: every player secretly picks one card from its hand, all players reveal their cards together and pass the rest of their hands on, and after three rounds the highest score wins. Klang et al. [4] introduced the game to TAG; the current version shows an agent only the cards it has seen.

Four properties make the game hard for a search agent. (i) The opponents' hands are hidden until they are passed on. (ii) Moves are simultaneous: no card can be chosen in reaction to an opponent's card of the same turn. (iii) Many points are delayed and conditional – sets pay out only when completed, Maki rolls at the end of the round and Pudding at the end of the game – so short simulations miss them and long ones are noisy. (iv) Hands of nine (three players) or eight cards (four players) give 729 or 4 096 joint moves per turn (more with Chopsticks), with 0.1–1 s per decision. TAG's BasicMCTS addresses none of the first three: it searches the one shuffled sample of the hidden cards it receives as if it were the true state, treats the players of a turn as moving one after another and evaluates random rollouts by the player's own score.

We ask which adaptations of MCTS to these properties make an agent stronger (RQ1), and how their value changes with the time per decision (RQ2) and the number of players (RQ3). We extend BasicMCTS in steps, keeping every modification as a switch, and test each in experiments of 500 games per condition (250 in some runs at 900 ms and in four-player development at 500 ms). Our contributions are:

- a model of *Sushi Go!*'s scoring that estimates every card's marginal value, used as the rollout policy of all players, and every player's final score, used in a bounded, lead-based evaluation;
- its combination on BasicMCTS with round-end rollouts, max-n backups, decoupled UCT and information-set search, compared with TAG's own decoupled information-set MCTS;
- exploratory evidence that settings tuned for four players hurt with three, and a diagnosis of why time beyond 500 ms helps little with four (a wide, shallow joint-move tree);
- leave-one-out and factorial ablations, held-out validation and negative results that locate the agent's remaining limits.

II. Background

A. TAG and Sushi Go!

TAG [1] splits a game into a state, a forward model and parameters; an agent receives a copy of the state, in which every card it has not seen is reshuffled, and uses the forward model to simulate moves. In TAG's *Sushi Go!* the 108-card deck holds Tempura (5 points per pair), Sashimi (10 per triple), Dumplings (1, 3, 6, 10 or 15 points for one to five), Salmon, Squid and Egg Nigiri (2, 3 and 1 points, tripled on an earlier Wasabi), Maki rolls with one to three icons (6 and 3 points for the most and second-most icons of a round), Chopsticks (two cards in a later turn) and Pudding (+6 for the most and, with three or more players, −6 for the fewest at the end of the game, when Puddings also break ties). Each round deals $12 - n$ cards to each of n players. As the hands rotate, every hand of a round has been seen after $n - 1$ passes, so hidden information is concentrated at the start of each round, while the value of a card depends on what the others will take from the hands we pass on.

B. Monte Carlo Tree Search

MCTS [2] builds a search tree by repeating selection, expansion, simulation (rollout) and backpropagation. Upper Confidence bounds applied to Trees (UCT) [5] selects the child that maximises the upper confidence bound (UCB1) $Q + K\sqrt{(\ln N / n)}$, where Q is the child's mean reward, n its visits and N those of its parent. Domain knowledge can enter through heuristic rollout policies or priors such as progressive bias [2], but a stronger rollout policy does not guarantee better estimates: Silver and Tesauro [6] showed that rollouts should be balanced, i.e., give unbiased value estimates.

C. Hidden Information, Simultaneous Moves and Several Players

Determinisation, or perfect-information Monte Carlo (PIMC) sampling, samples the hidden information, searches each sample as if it were the true state and combines the

All sections, tables and figures of this paper were drafted with Claude [15]; the prompts are given in the statement on the use of generative AI at the end of the paper.

results. Long et al. [7] related its success in card games to properties such as how quickly hidden information is revealed during play, which in *Sushi Go!* happens within $n - 1$ turns. Its known flaws are strategy fusion – the search assumes that it can act differently in states it cannot tell apart – and non-locality [8]. Information-set MCTS (ISMCTS) [3] avoids strategy fusion by searching one tree over the searcher's information sets and sampling the hidden information anew in every iteration. In its single-observer form the opponents' choices in the tree are not conditioned on their own private information; the multiple-observer form models it with one tree per player at a higher cost [3], an idea TAG's advanced MCTS agent offers as MultiTree MCTS [9].

Simultaneous moves are usually handled by decoupled UCT (DUCT): every player chooses its own action at a node from its own bandit statistics and the joint action leads to a child [10]. UCB-based DUCT need not converge to an equilibrium, so Lanctot et al. [10] compared it in Goofspiel with EXP3, regret matching and their online outcome sampling: regret matching and outcome sampling played best, and decoupled UCT's strategies were the most exploitable. Tak et al. [11] found across nine games that the best variant depends on the game and the opponent, although decoupled UCT was best overall. With several players, UCT with max-n backups (every player maximises its own value) computes a mixed-strategy equilibrium and played at least as well as earlier algorithms [12].

MCTS has also been applied to the drafting game 7 Wonders, which passes hands in the same way, and many results known from classic games held there too [13]. TAG's advanced MCTS agent already offers information sets, one tree per player and a decoupled option; we build on BasicMCTS, which has none of them, so that the contribution of each idea can be measured. We know of no study that separates the contributions of decoupled selection, information sets and domain knowledge within one agent or measures how they change with the time budget and the number of players; the earlier *Sushi Go!* study in TAG [4] focused on simultaneous action selection and complete information and concluded that better heuristics should improve the agents, which motivates our card-value model.

III. Method

Our agent, SushiMCTS, is a modified copy of TAG's BasicMCTS (package *players.basicMCTS*) and uses no code of TAG's advanced MCTS agent. Every modification is a switch in its parameter class (Table I). We compare the final agent (v4) with two earlier versions, v2 (H, A, B, D) and v3 (H, B, D, P, R, M), whose tree treats the players of a turn as moving in sequence (Fig. 1). The search techniques come from the literature; our own element is the card-value model shared by the evaluation and every player's simulation.

A. Card-Value Model and Heuristic (H)

For a state s and player p the model estimates the final score

$$\hat{S}_p = b_p + \alpha u_p + m_p + \pi w_t\, d_p, \tag{1}$$

where b_p is the banked score (TAG scores Tempura, Sashimi, Dumplings and Nigiri when they are revealed); u_p the value of unfinished sets, of unused Wasabi (2ν, the bonus for a Nigiri of average value $\nu = 2$) and of Chopsticks (1), each weighted by $\min(1, (r - k + 1)/T)$, or 0 if $r < k$, when k more cards are needed and r picks are left; m_p the Maki points p would get if the round ended now; and d_p the Pudding (dessert) points if the game ended now, weighted by $w_t = t/3$ in round t ($\alpha = 0.5$, $T = 4$, $\pi = 1$). The value of s for player i is

$$H_i(s) = \tanh(\Delta_i / \sigma), \tag{2}$$

$$\Delta_i = w(\hat{S}_i - \max_{j \neq i} \hat{S}_j) + (1 - w)(\hat{S}_i - \mathrm{mean}_{j \neq i} \hat{S}_j), \tag{3}$$

with $\sigma = 15$ and $w = 0.7$: the first term rewards beating the leader, since only first place wins, and the second keeps a gradient when the agent is far ahead or behind. The marginal value $mv_p(c)$ of a card c is the change of $\hat{S}_p$ when p plays it.

TABLE I. Modifications of BasicMCTS (✓ = used in the final agent with 3 / 4 players)

Id	Modification	3 p / 4 p
H	heuristic (2)–(3) from model (1)	✓ / ✓
B	ε-greedy card-value rollouts for every player	✓ / ✓
R	rollouts end when the round is scored	✓ / ✓
P	one branch per distinct card	✓ / ✓
M	max-n backups [12]	✓ / ✓
S	decoupled UCT, one node per turn [10]	✓ / ✓
I	single-observer information-set search [3]	✓ / ✓
X0	four-player card constants [a]	– / ✓
W2	win-gap value for 4 players (Section III-D)	– / ✓ [b]
F, L	fast simulator, 8 rollouts per leaf	– / ✓ [b]
A	progressive bias [2]	dropped in v3
D	ensemble of up to 4 determinised trees [3]	replaced by I

[a] $(\sigma, w, \alpha, T, \beta, T_m, \kappa, \delta, \pi, \nu, \gamma) = (15, 0.7, 0.5, 6, 0.5, 3, 0.25, 1.5, 1.25, 3, 3)$, default $(15, 0.7, 0.5, 4, 1, 0, 0, 0, 1, 2, 0)$. β scales a Sashimi triple still needing two cards; the Maki standing has weight $1/(1 + r/T_m)$ (1 if $T_m = 0$), κ per icon the rest; δ weights the next Dumpling; γ: cards needed per missing card. [b] 500 ms or more.

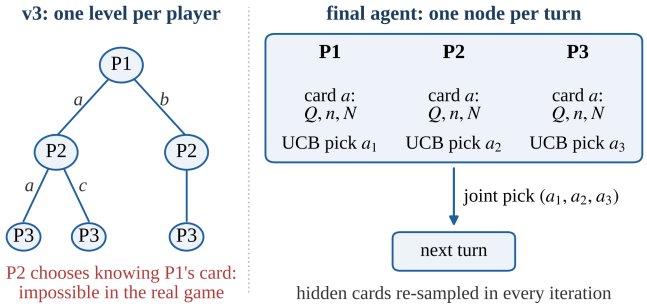


Fig. 1. The turn node of the final agent (right) and the sequential tree of v3 (left). The final agent re-samples the cards it cannot see in every iteration (I), lets every player pick by UCB on its own card statistics (S, P), plays the joint pick, rolls out with card-value policies until the round is scored (B, R) and backs every player's value (H) up to the card it chose (M); after the time budget it plays its most visited root card.

B. Simulation (B, R)

B: every player plays its card with the highest *mv*, or a random card with probability $\varepsilon = 0.2$ (the second card of a Chopsticks move is chosen in the same way); the same policy models the opponents. **R:** a rollout stops when the current round has been scored; only Pudding carries over, which (1) captures, so at the leaves $u_p = m_p = 0$ and only banked points and the Pudding standing count.

C. Tree (P, M, S, I)

P: identical cards lead to identical states, so a node has one branch per distinct card. **M:** a node stores one value per player and every player maximises its own value [12], instead of BasicMCTS's paranoid assumption that every opponent minimises ours. **S:** one node represents a whole turn; at a node every player p picks a card by

$$a_p = \mathrm{argmax}_a \left[Q_p(a) + K\sqrt{(\ln N_p(a) / n_p(a))}\right] \tag{4}$$

from its own statistics, trying untried cards first; the joint choice is played in the forward model and leads to the child of that joint action [10]; in v3's tree a later player "saw" the card an earlier player had just chosen (Fig. 1). **I:** every

iteration starts from a new sample of the cards we cannot see and walks one shared open-loop tree, whose nodes store statistics but no states [3]. Statistics are keyed by card type; $n_p(a)$ counts how often p chose a at the node and the availability count $N_p(a)$ how many iterations reached the node with a in p's sampled hand. I replaces the determinisation ensemble D of v2 and v3, which split the budget between up to four independent trees. With three players $K = 0.5$.

D. Four Players

With four players the agent switches parameters. It uses the card constants X0 (Table I), found by a search-free tuner of greedy card-value policies. From 500 ms it also uses **W2** – $w = 1$; finished games valued by $\tanh((b_i - \max_{j \neq i} b_j)/2)$ with Pudding as tie-break; σ multiplied by $(\rho/2)^{0.5}$ for ρ rounds left; opponents in the backup valuing their lead over the others' mean – and **F**, TAG's rules re-implemented on integer arrays and checked move by move against the forward model, whose speed is spent on **L** = 8 rollouts per leaf, with $K = 1$.

E. Experimental Setup

TABLE II. EXPERIMENTAL SETUP

Item	Setting
Framework	TAG as provided for the module, *Sushi Go!* with its default rules; OpenJDK 21
Opponents	v2: our previous agent (H, A, B, D, K 1.41). TAG ISMCTS: SushiGo_3+P_HiBudget at our budget (one tree per player [9], UCB-Tuned, K 1, random rollouts, score heuristic). TAG DUCT: SushiGoDUCT (decoupled UCT, K 1, information sets, lead heuristic, 30-move rollouts). BasicMCTS (K 1.41, 10-move rollouts), Rolling Horizon Evolutionary Algorithm (RHEA; horizon 10, population 10), One-Step Look-Ahead (OSLA, score heuristic)
Budget	CPU time per decision, equal for all agents: 100 ms (most development), 500 and 900 ms; submitted agent files: 900 ms (1 s limit) or 430 ms (500 ms limit), with wall-clock guards
Seating	our agent in seat g mod n in game g; the other seats alternate between the two opponents (4 players: 2 + 1, swapped every 4 games)
Deals	game g uses seed $s + g$; from 19 October (all seed-7 and held-out runs) a fixed deck order is shuffled with it (equal s, equal deals: paired runs); earlier runs used TAG's own deal (independent samples)
Seeds	development s = 1, 2 (v3 also 1001, 2001, 3001); ablation and later candidate tests s = 7; held-out s = 100 000, disjoint from all other deals
Games	500 per condition; 250 at 900 ms (development, s = 7) and in four-player development at 500 ms (45–70 s of CPU time per game)
Hardware, records	18-core laptop (6–16 games in parallel); candidate tests and validation on a 4-core machine (4 in parallel). Agent files, per-game results and scripts: from the authors

Table II summarises the setup. Each game is an independent trial whose outcome for our agent is 1 for a win, $1/k$ for a k-way shared first place and 0 otherwise. Win rates are reported with Wilson 95% confidence intervals (CI) [14]; the score margin (our score minus the best opponent's) as mean ± 1.96 standard errors (SE); differences between runs as the mean of the per-game differences on the same deals ± 1.96 SE (for independent samples, the difference of the means with the two-sample SE) and z = difference/SE, in percentage points (pp) for win rates. With 500 games a win rate near 50% has a 95% interval of ±4.4 pp, and a difference between two runs about ±5.7 pp even when paired, because the outcomes of two agents on the same deal correlate only weakly (0.0–0.2); such a comparison detects a difference of about 8 pp with 80% power. The score margin, usually the more sensitive measure (z values 1.0–2.3 times as large in the ablation), served to screen ideas. We report unadjusted intervals, apply a Holm correction within the ablation and within the later candidate tests, and treat the four-player comparisons (59 of our 163 runs) as exploratory. We expected (H1) every component to improve both measures, (H2) the decoupled information-set tree to matter more at longer budgets and (H3) the best settings to depend on the number of players; H2 arose from the development runs and is tested on the held-out deals.

IV. RESULTS

A. Final Agent (RQ1)

TABLE III. FINAL AGENT ON HELD-OUT DEALS AND AGAINST OTHER AGENTS

Setting	Opponents	*n*	Win % [95% CI]	Margin
3p/100	v2, ISMCTS	500	52.6 [48.2, 56.9]	+0.73 ± 0.46
3p/500	v2, ISMCTS	500	47.2 [42.9, 51.6]	−0.10 ± 0.40
3p/900	v2, ISMCTS	500	pending	pending
4p/100	v2, ISMCTS	500	44.4 [40.1, 48.8]	−0.61 ± 0.43
4p/500	v2, ISMCTS	500	pending	pending
3p/100	DUCT, ISMCTS	500	55.0 [50.6, 59.3]	+0.72 ± 0.40
3p/100	ISMCTS × 2 [a]	500	51.3 [46.9, 55.7]	+1.41 ± 0.51
3p/100	BasicMCTS, RHEA [a]	500	84.2 [80.7, 87.1]	+6.68 ± 0.60
3p/100	OSLA, BasicMCTS [a]	500	87.2 [84.0, 89.8]	+9.74 ± 0.75

Setting: players / ms per decision. Deals: held-out seed block 100 000; [a] seed block 7. Margin: our score minus the best opponent's in points, ± 95% CI. Chance: 33.3% (3 players), 25% (4 players).

Table III reports the final agent: on the held-out deals it wins 47.2–52.6% of three-player and 44.4% of four-player games against v2 and TAG's ISMCTS. On seed block 7, whose deals overlap the development deals, it had won 55.0% at 100 ms (58.1% when re-run on the machine of the held-out runs), 2.4 ± 6.1 pp and 5.5 ± 6.1 pp more than on the held-out deals, neither significant. TAG's DUCT agent, with decoupled selection and information sets but neither our card-value model nor round-end rollouts, wins 21.0% of its seats when it replaces v2, the same share as v2. BasicMCTS, RHEA and OSLA win at most 12.8% of their seats; OSLA none, as its one-step look-ahead never reaches the simultaneous reveal.

B. Ablation (RQ1)

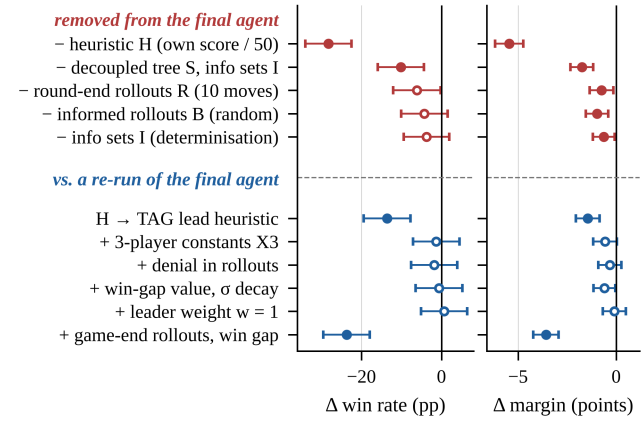


Fig. 2. Paired effects with three players at 100 ms (seed block 7, 500 games each), ± 95% CI; filled markers survive a Holm correction within their group. Top: one component removed from the final agent (base: final agent, 55.0%). Bottom: TAG's lead heuristic and five later candidate changes, against a re-run of the final agent on the machine of the held-out runs (58.1%).

Fig. 2 (top) removes one component at a time from the final agent. Every removal lowers both measures; on the

score margin all five differences survive a Holm correction, on the win rate only those of H and of S + I (−R: $z = 2.0$, −B: 1.4, −I: 1.3), so H1 holds for the margin and only partly for wins. Without H the leaves are valued by TAG's default heuristic (own score / 50); as K was not retuned to its scale, the 28.3 ± 5.8 pp loss mixes the value of H with mis-scaled exploration. TAG's lead heuristic (own score minus the best opponent's) with K rescaled to 7.5 costs 13.6 ± 5.9 pp ($z = 4.5$): valuing the lead recovers roughly half of H's effect (two estimates against different base runs). As round-end leaves hold no unfinished sets, H differs from it only by the Pudding standing, the mean-opponent term and its bounded scale, which were not separated; the card values act through the rollout policy B. S and I together are worth 10.2 ± 5.7 pp ($z = 3.5$). S alone, decoupled selection with v3's determinisation, beats v3 by +6.4 ± 5.7 pp ($z = 2.2$), and I adds a further 3.8 ± 5.7 pp, a small gain, as Long et al.'s analysis [7] suggests for a game that reveals its hidden information within $n − 1$ turns. P and M cannot be removed, but a 2^4 factorial on v3 (R; M; $K = 0.5$ vs 1.41; completion horizon $T = 8$ vs 4; 16 × 500 games) gave main effects of +8.7 pp for R ($z = 8.1$), +5.2 for K, +4.1 for M and +3.6 for $T = 8$ (each ±2.1); of the six interactions only R × T (−4.2: with R, $T = 8$ no longer helps) and M × K (+3.2) exceed $|z| = 2$.

Of five later candidate changes (Fig. 2, bottom), four changed the win rate by at most 1.8 pp (each ±5.8), among them card constants tuned for three players (X3), a stronger greedy policy (45.0% of greedy games against two X0 players); rollouts to the game end with a win-gap value cost 23.7 ± 5.8 pp ($z = 8.0$), the only change that survives a Holm correction. As X3 also changes the Pudding weight of the leaf evaluation, its null result only weakly echoes simulation balancing [6].

C. Time Budget (RQ2)

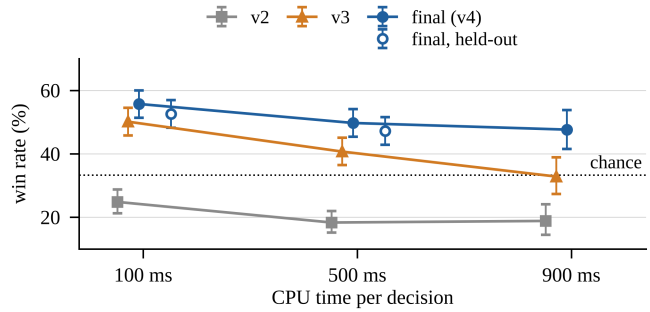


Fig. 3. Win rate with three players against v2 and TAG's ISMCTS by CPU time per decision. Lines: development runs (seed 1, independent samples; 250 games at 900 ms). Open markers: the final agent and v3 on the held-out deals (seed block 100 000). Bars: Wilson 95% intervals.

In development runs (Fig. 3; independent samples) v3 doubled v2's win rate at 100 ms (24.8% → 50.1%) but lost 17.3 ± 7.2 pp from 100 to 900 ms, the final agent 8.1 ± 7.5 pp; the gain from S and I rose from +5.6 ± 6.1 pp to +14.8 ± 8.4 pp, by +9.2 ± 10.4 pp ($z = 1.7$). We hypothesise that v3's decline comes from its sequential tree, in which later players react to cards chosen earlier in the same turn: a wrong model that more iterations exploit more. S alone gained +1.3 ± 6.1 pp at 100 ms but +6.2 ± 6.1 pp at 500 ms, and v3 and the final agent ran similar numbers of iterations (11 170 and 11 865 per decision at 500 ms). As v3's 50.1% at 100 ms was the best of the 16 factorial cells (45.1–49.4% on three new seed sets), the development rise is probably overstated. On seed block 7 the final agent won 55.0%, 49.4% and 56.0% at 100, 500 and 900 ms: its development decline did not replicate. The final agent's own win rate on the held-out deals did not change significantly with the budget (−5.4 ± 5.9 pp at 500 ms against 100 ms, same deals), but its score margin fell by 0.83 ± 0.58 points. Much of the fall of the curves comes from the opponent: TAG's ISMCTS won 23.1% of its seats against the final agent at 100 ms but 37.4% at 900 ms (against v3: 33.0% → 46.0%).

D. Four Players and the Limits of Search (RQ3)

In four-player development no change gave a significant gain at its own budget: X0 added +4.3 ± 4.3 pp at 100 ms (two runs of each pooled), X0 with W2 and $K = 1$ added +5.8 ± 7.4 pp at 500 ms (two runs pooled) and F + L a further +1.0 ± 7.9 pp, whereas F + L cost 6.6 ± 6.0 pp at 100 ms; we kept each change where its estimate was positive. With three players at 500 ms (both arms with F + L), X0, W2 and $K = 1$ cost 7.7 ± 5.4 pp ($z = 2.8$), an effect 13.5 ± 9.1 pp ($z = 2.9$) different from that with four. This supports H3, although the gains with four players were not significant; as the contrast was chosen after the four-player runs, it is exploratory. Depth matters – at 500 ms a tree cut to depth 1 lost 17.4 ± 8.2 pp – and so does time up to 500 ms: with 100 ms (and its 100 ms settings) our agent won 9.0 ± 7.7 pp less ($z = 2.3$), but 1 000 ms (with $K = 1.37$) gave −4.8 ± 8.1 pp. On 73 positions from random games the 500 ms tree has 704 joint-move children at the root on average, 53% of them visited once (mean depth 1.5); doubling the time or the rollouts per leaf changed the card in 2 and 4 of them, no more often than a re-run with the same seed (4) or another seed (7).

V. Conclusions and Future Work

For RQ1, removing any component lowers the final agent's score margin significantly, but after a Holm correction only removing the evaluation H (worth 13.6 pp over TAG's lead heuristic) or the decoupled information-set tree (10.2 pp over v3's sequential, determinised tree) also lowers its win rate. On held-out deals the final agent wins 52.6%, 47.2% and [pending] of three-player games at 100, 500 and 900 ms against TAG's ISMCTS and our previous agent, and 44.4% of four-player games at 100 ms. For RQ2, in development v3's sequential tree lost more strength at longer budgets than the decoupled one, though the difference between the trends is not significant. For RQ3, settings tuned for four players cost 7.7 ± 5.4 pp with three players while their gain with four was not significant (an exploratory result), and with four players time beyond 500 ms rarely changes the decision of the wide, shallow joint-move tree.

Our opponents were framework agents and our own previous version, the four-player settings rest on 250-game runs and non-significant gains, and CPU-time budgets hide wall-clock effects. As hand-tuned changes to the model no longer move the agent, future work should learn the evaluation from game outcomes, e.g. by regressing final ranks on features of round-end states; regret matching or EXP3 deserve a test, as decoupled UCT is best overall but not in every game [11] and was the most exploitable in Goofspiel [10]; and opponent models fitted to observed picks could replace the single greedy policy that now models all players.

Use of Generative AI

We used Claude [15] (Anthropic), through the Claude Code tool, throughout the project: to write and modify the Java code of our agent on top of BasicMCTS, to write and run the experiment and statistics scripts, and to draft all sections, tables and figures of this paper from the logs of our runs; every number is computed from the result files of our

runs. The authors take full responsibility for the code and the paper, including any errors. The paper was generated with the prompt (original Turkish, then English translation): *"simdi reponun icindeki butun herseyi incele ve hocaya direk yuklyecegimiz raporuda olusutr repoda ornekler tipler brief hersey var 100 alcagimiz bir rapor olsun her raporu tamaladiginda baska bir agent spawn edip bagimsiz durusstce kontrol taki yuzde yuz alacagana emin olankadar looptan cikma"* – "Now examine everything in the repository and also create the report we will upload directly to the lecturer; the repository has examples, tips, the brief, everything. It should be a report that gets 100. Every time you finish the report, spawn another agent to check it independently and impartially, and do not leave the loop until you are sure it will get 100%." Following that prompt, each draft was marked by a second, independent Claude instance, prompted as "an independent, strict marker for the QMUL module ECS7032P" to mark it against the brief and the rubric, to recompute its numbers from our result files and to list the changes that would raise the mark. Its written reports, one per draft (available on request), were the prompts for the revisions; they asked, among other things, for validation on deals disjoint from development, a correct description of the availability-count UCB, all replicate runs, intervals and hedges for non-significant trends, the comparison with TAG's DUCT agent, the lead-heuristic ablation, v3 on the held-out deals, the four-player diagnostics, correct figure references, this statement's citation and prompts, claims no stronger than the statistics and the selection bias of v3's development run. The later candidate changes were tested with the prompt "The lecturer will hold a tournament of our agents in class; as before, improve the agents' performance quickly while following the lecturer's restrictions and recording what we did in the README; you have 4.5 hours, test everything in parallel." The prompts of earlier development sessions are not reproduced here; they asked Claude to implement and test the modifications of Table I and to run the experiments of Section IV.

REFERENCES

[1] R. D. Gaina, M. Balla, A. Dockhorn, R. Montoliu, and D. Perez-Liebana, "TAG: A tabletop games framework," in *Proc. AIIDE Workshop on Experimental AI in Games (EXAG)*, 2020.

[2] C. B. Browne *et al.*, "A survey of Monte Carlo tree search methods," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, 2012.

[3] P. I. Cowling, E. J. Powley, and D. Whitehouse, "Information set Monte Carlo tree search," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 2, pp. 120–143, 2012.

[4] C.-M. E. Klang, V. Enhörning, A. Alvarez, and J. Font, "Assessing simultaneous action selection and complete information in TAG with Sushi Go!," in *Proc. IEEE Conf. Games (CoG)*, 2021, pp. 1–4.

[5] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *Proc. 17th Eur. Conf. Mach. Learn. (ECML)*, 2006, pp. 282–293.

[6] D. Silver and G. Tesauro, "Monte-Carlo simulation balancing," in *Proc. 26th Int. Conf. Mach. Learn. (ICML)*, 2009, pp. 945–952.

[7] J. Long, N. R. Sturtevant, M. Buro, and T. Furtak, "Understanding the success of perfect information Monte Carlo sampling in game tree search," in *Proc. 24th AAAI Conf. Artif. Intell.*, 2010, pp. 134–140.

[8] I. Frank and D. Basin, "Search in games with incomplete information: A case study using Bridge card play," *Artif. Intell.*, vol. 100, no. 1–2, pp. 87–123, 1998.

[9] J. Goodman, D. Perez-Liebana, and S. Lucas, "MultiTree MCTS in tabletop games," in *Proc. IEEE Conf. Games (CoG)*, 2022, pp. 292–299.

[10] M. Lanctot, V. Lisý, and M. H. M. Winands, "Monte Carlo tree search in simultaneous move games with applications to Goofspiel," in *Computer Games (CGW 2013)*, Commun. Comput. Inf. Sci., vol. 408. Cham, Switzerland: Springer, 2014, pp. 28–43.

[11] M. J. W. Tak, M. Lanctot, and M. H. M. Winands, "Monte Carlo tree search variants for simultaneous move games," in *Proc. IEEE Conf. Comput. Intell. Games (CIG)*, 2014, pp. 232–239.

[12] N. R. Sturtevant, "An analysis of UCT in multi-player games," in *Computers and Games (CG 2008)*, Lect. Notes Comput. Sci., vol. 5131. Berlin, Germany: Springer, 2008, pp. 37–49.

[13] D. Robilliard, C. Fonlupt, and F. Teytaud, "Monte-Carlo tree search for the game of '7 Wonders'," in *Computer Games (CGW 2014)*, Commun. Comput. Inf. Sci., vol. 504. Cham, Switzerland: Springer, 2014, pp. 64–77.

[14] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[15] Anthropic, "Claude" [large language model, used through the Claude Code tool], Oct. 2026. [Online]. Available: https://claude.ai (accessed Oct. 28, 2026).